

COMP101
Introduction to Programming
2025-26

Week-01 Lab-03
Drop-In – no supervision

Workbook-03
Python Command Shell
‘Shell’, ‘Python Shell’, ‘Python Standard Shell’

REMEMBER:
THE IDLE EDITOR IS WHERE WE PROGRAMME
Don't Program in the Shell

Don't use the shell for assignment submissions.

The shell is known by the general term REPL (**R**ead-**E**valuate-**P**rint-**L**oop) which basically describes how it operates.

It uses a prompt (>>>) to process:

Read – an input

Evaluate – assess input for errors

Print – output an answer or an error message

Loop - returns the user to the prompt ready for the next input

Input can be numbers or strings and output can be the answer or error messages.

Using the shell:

Python is an 'interpreted language' – it converts our English input into a language the computer can understand.

The shell gives access to the interpreter for immediate processing so we can run (execute) code line-by-line interactively to give an immediate answer.

This functionality can help identify coding problems – but note it is not a substitute for IDLE and it is not a debugger.

Don't use the shell for assignment submissions.

Ex3.1

Open Python and the shell window appears.

The shell window looks like figure-1 below - there may be minor differences depending on the version of Python.

Python IDLE Environment

- Python Shell Window – first thing to be seen on screen

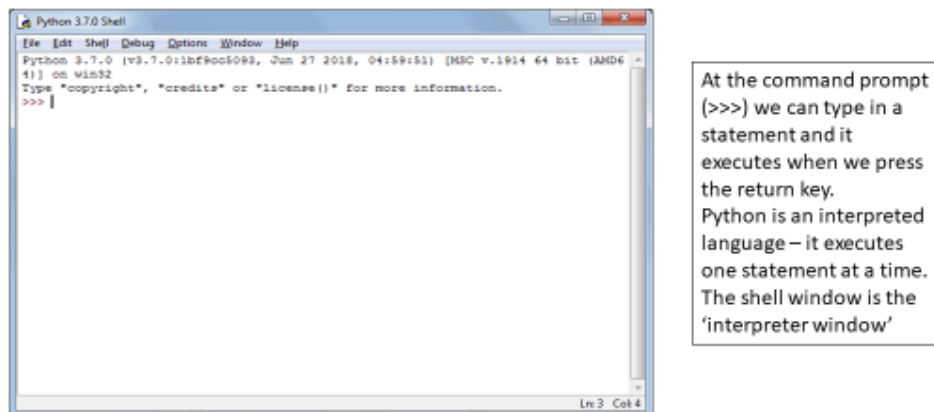


Figure-1:

The shell is part of the Python IDLE environment – the other part of interest to us is the ‘script editor’ – see workbook-02.

There is a banner at the top telling us what version we are using and legalities. Underneath the banner is where we type in commands at the ‘command prompt’

All communication is via three chevrons (`>>>`)

Depending on the installation, this may show as a numbered line, e.g. `In [1]` :

Enter a command, then press Return to see the result.

- a) `>>>` input (command) prompt
- b) The prompt disappears and we get output as a result of evaluating the input
- c) `>>>` Prompt re-appears ready to accept more input

Note using a `#` symbol at the prompt means it is a comment and will be ignored

The interaction is limited, but this is a useful environment to try out ideas that you want to put in your code or to check if your calculations are correct. It may be useful to know a little about the shell as some authors on web sites use it to demonstrate programming.

It is also useful if you want to force an error, as understanding error messages greatly increases the chances of successful debugging. As with most high level languages, error messages appear a bit cryptic at first until we learn how to decipher them to understand what they are actually telling us.

When we program, what we type in will be defined as a ‘data type’. Data types have categories – broadly ‘numeric’ and ‘string’

Numeric Types

integer:	2, 3 etc	(no decimal part)
real (or ‘float’)	3.142 etc	(have a decimal part)

String (or ‘str’) Types

There is no character (‘char’) type in Python

string: ‘cat’, ‘dog’, ‘parrot’, etc

string: ‘a’, ‘b’, ‘c’ etc (other languages use ‘character’ (‘char’))

(These will be extended when we come to do programming – these will do for now while we learn the shell).

The shell is useful to test some ideas, but is not suitable for full coding of a problem, and does not really give much programmer-support to help us program a problem.

Ex1

Input numeric commands at the command prompt

Actual arithmetic and outcomes will be discussed in lectures.

These are just exercises to practice with the shell to see what we get from the interpreter when using standard arithmetic operators:

addition(+), subtraction (-), multiplication (*) and division (/).

(i)

```
>>> 2+2
```

Type in 2+2 and press return

Integer 4 outputs on a new line and you are returned to the command prompt

Observation:

```
>>>2+2
```

#the 'read' part of REPL
#after reading the command in, it is evaluated for correctness
(the 'evaluate' part of REPL)

4

#the 'print' part of REPL – output the answer if the
evaluation is correct

Error message

#the 'print' part of REPL – output an error message if the
evaluation is incorrect

```
>>>
```

#the 'loop' part of REPL – returns us to the command
prompt so we can do it again

Anything beginning with a hash (#) is a 'comment' and is ignored by Python.

The up-arrow recalls the previous command(s) if you wanted to type the same command in again.

ii)

Try combinations under the Input (>>>) column:

... and try some of your own.

Input (>>>)	Output (expected)	Output(actual)	Comment
5-2		3	
1-1		0	
6-9		-3	
3.142 * 2		6.284	o/p to 3dp
3.142 * 2.5		7.8549999999999995	o/p to 16dp
3.142*4.5		14.139	o/p to 3dp
30/5		6.0	
30.5/5.5		5.545454545454546	
30.5/2		15.25	
32/2.5		12.8	
Try your own			

(iii)

>>> 2>3

Type in 2>3 and press return

'False' outputs and you are returned to the command prompt.

You have asked the interpreter to evaluate if 2 is greater than 3

It also works if you put spaces into the input

(iv)

>>> 3>2

Type in 3>2 and press return

'True' outputs and you are returned to the command prompt.

You have asked the interpreter to evaluate if 3 is greater than 2.

It also works if you put spaces in your input.

How many decimal places do we need?

NASA site at:

<https://www.jpl.nasa.gov/edu/news/2016/3/16/how-many-decimals-of-pi-do-we-really-need/>

Ex2

Input string commands at the command prompt

In Python, words and single characters are called 'strings'.

Python is case-sensitive – a common error when starting to program in Python.

To output a string, we need the 'print' command followed by the string to be output. The string must be inside parentheses and surrounded by apostrophes.

(i)

```
>>>print ('a')
```

The string 'a' is output and you are returned to the command prompt.

It will output without the apostrophes: The apostrophes and parentheses are there to avoid a 'syntax error'

(ii)

```
>>>print ('Hello world')    #one space between the words
```

String 'Hello world' is output and you are returned to the command prompt.

It will output without the apostrophes: The apostrophes and parentheses are there to avoid a 'syntax error'

(iii)

```
>>>print ('Hello   world')    #5 spaces between the words
```

String 'Hello world' is output with no apostrophes and you are returned to the command prompt.

There are five spaces between the words – it shows that anything between the apostrophes will output exactly as it was typed in.

is a comment – it is ignored by the interpreter

(iv)

```
>>>print ("Hello   world")    #5 spaces between the words
```

We have used different apostrophes. Python allows us to use either single or double apostrophes.

You should see the words 'Hello world' are output and you are returned to the command prompt.

Python is helpful in separating the input/output for readability and for tracking errors. Have a look at the command shell and its use of colours.

Output appears in blue, and other colours are used depending on what we are typing in.

Errors appear in red - used for error messaging.

Ex3

Error messages

Error messages output in red. Different messages can appear depending on the nature of the error.

(i)

```
>>>2+a
```

This gives an error message:

This is one of those error messages that is confusing at first sight, but it basically means we can't add strings and numbers together.

Understanding what the error message actually means comes with practice.

Traceback (most recent call last):

File "<pyshell#0>", line 1, in <module>

#may say File "<stdin>"

2+a

NameError: name 'a' is not defined

(ii)

```
>>>a
```

This gives an error message: It means Python doesn't know what you want to do with the input.

Traceback (most recent call last):

File "<pyshell#0>", line 1, in <module>

NameError: name 'a' is not defined

In both of the above inputs, Python outputs an error message telling us where the error is and what type of error it is:

It is in the command shell:

File "<pyshell#0>" (or it may say "<stdin>");

It is in line 1:

line 1 (we have only typed in one line, so it is in line number 1);

It is in the named module:

in <module>

(i.e. what program it is in – as we haven't actually saved anything as a named program, Python defaults the filename to <module>);

It is (finally) giving us the type of error:

NameError: name 'a' is not defined

This one is a 'NameError' (there are other error names)

(iii)

```
>>> print ("a')
```

This outputs an error because we have mixed the quotes.

Python is using an error message called 'SyntaxError'.

Python finds the first quote (a double quote) and expects to find another double quote (effectively closing the string).

But it finds a single quote. Now it goes looking for another single quote (as it thinks this single quote is nested with two double quotes), but it doesn't find it.

Basically it 'falls off' the end of the input line having unsuccessfully closed the string:

It is in the command shell:

File "<pyshell#0>" (or it may say "<stdin>")

It is in line 1:

We haven't typed in any other lines

It shows us the error code:

```
print ("a')  
          ^
```

It is (finally) giving us the type of error:

SyntaxError: EOL while scanning string literal

This one is a 'SyntaxError'

It is expecting the single apostrophe to be followed by a double one but it finds a closing parenthesis instead.

SyntaxError usually means we have mis-typed something or put something in the wrong order.

The EOL (End Of Line) message indicates Python has finished reading the entire line of code and cannot move on to any other lines (if there were any) as it has found an error in what it has just read in.

It is why the circumflex (^) is positioned at the end of the line after the last parenthesis.

You will find when we program using IDLE, this type of message may look like it has occurred on the next line of code when in fact the error may be in the line above it;

i.e. Python discovers the error when it has read in the whole of the line containing the error and has begun to read the next line but has to stop until the error in the line above has been fixed.

(iv)

```
>>>Print ('a')
```

This will give a 'NameError' message:

Python is case sensitive – the word 'Print' must be all lower case

Traceback (most recent call last):

File "<pyshell#0>", line 1, in <module>

Print ('a')

NameError: name 'Print' is not defined

Ex4

Using variables (More on this in lectures)

We want to use a single data value;

A data value has a 'type' (see Exercise 2.1);

The 'data type' can be one of:

Integer, real or string

Regardless of the data type, we can store the data in a 'variable' ('var') for use in our programming.

The name of the variable is chosen by the programmer.

(i)

```
>>> number = 2          # 'assigns' integer value 2 to variable name 'number'
```

```
>>> number
```

```
2
```

Observation:

Integer type 2 is output

(ii)

```
>>> pi = 3.142
```

```
>>> pi
```

```
3.142
```

Observation:

Real type 3.142 is output

(iii)

```
>>> pi = 3.14159
```

```
>>> pi
```

```
3.14159
```

Observation:

Real type 3.14159 is output

The 'new' value 3.14159 overwrites the 'old' value of 3.142 from (ii) above

(iv)

```
>>> number + pi          # i.e. calculate 2 + 3.14159
```

```
5.14159
```

Observation:

5.14159 is output

The values held in the variables are 'remembered' and can be used in calculation. Remember – the var name must be exactly the same as the original

(vi)

```
>>>pi+value
```

Traceback (most recent call last):

File "<stdin>", line 1, in <module>

NameError: Name 'valu' is not defined

Observation:

Error type 'NameError' is output.

The original variable name was 'number' but in the calculation we used 'value'

(v)

```
>>>Superhero = "Batman"
```

```
>>> name
```

```
'Batman'
```

Observation:

This is a 'string data type' assigned to a named variable 'Superhero'.

The data type is a string in double quotes – output is the string in single quotes.

It doesn't matter if you put a string in single or double quotes, it outputs in single quotes

(vi)

```
>>>x = 5
```

```
>>>y = 2.5
```

```
>>>x * y
```

```
12.5
```

Observation:

12.5 is output. The 'real variable', y takes precedence over the 'integer variable', x so the output is a real value. There are no quotes around a numeric variable

Ex5

help()

Pass a type, method, or other object to help to instantly see available documentation about the method, the parameters it expects, and what it returns.

(i)

>>> **help ()** #remember the parentheses

Observations:

Calls the help function which gives a brief explanation of what it does;

Prompt changes to **help>**

Tricky to read until you get used to it - usually quicker and easier to seek help on a web search – but it can come in handy within the shell

(ii)

help> quit #or q

Takes us out of the help() function and back to the command prompt (>>>)

(iii)

>>> **help()**

help> print

Observation:

Press return and the help page for the print command appears

Try a few others – but don't spend too much time on it

Ex6

type()

We can find the ‘type’ of data the variable is holding

Assign the data to a variable

Pass the variable name into the type() command

(i)

```
>>> a = 2
```

```
>>> type(a)
```

Observation:

<class 'int'> is output

The type data held in the variable is ‘int’ – integer type

This makes variable a an ‘integer variable’

(ii)

```
>>> b = 3.142
```

```
>>> type(b)
```

Observation:

<class ‘float’> is output

The data type held in the variable is a ‘float’ type , i.e. a real number

This makes variable b a ‘float (or ‘real’) variable’

(iii)

```
>>> c = ‘me’
```

```
>>> type(c)
```

Observation:

<class ‘str’> is output

The data type held in the variable is a ‘str’ type , i.e. a single character or more than one character

This makes variable c a ‘str (or ‘string’) variable’

Ex7

dir()

Show all the methods that can be used with a type:

(i)

```
>>>dir(int)
```

Observation:

```
['__abs__', '__add__', '__and__', '__bool__', '__ceil__', '__class__', '__delattr__',  
 '__dir__', '__divmod__', '__doc__', '__eq__', '__float__', '__floor__',  
 '__floordiv__', '__format__', '__ge__', '__getattr__', '__getnewargs__',  
 '__gt__', '__hash__', '__index__', '__init__', '__init_subclass__', '__int__',  
 '__invert__', '__le__', '__lshift__', '__lt__', '__mod__', '__mul__', '__ne__',  
 '__neg__', '__new__', '__or__', '__pos__', '__pow__', '__radd__', '__rand__',  
 '__rdivmod__', '__reduce__', '__reduce_ex__', '__repr__', '__rfloordiv__',  
 '__rlshift__', '__rmod__', '__rmul__', '__ror__', '__round__', '__rpow__',  
 '__rrshift__', '__rshift__', '__rsub__', '__rtruediv__', '__rxor__', '__setattr__',  
 '__sizeof__', '__str__', '__sub__', '__subclasshook__', '__truediv__', '__trunc__',  
 '__xor__', 'bit_length', 'conjugate', 'denominator', 'from_bytes', 'imag',  
 'numerator', 'real', 'to_bytes']
```

The list of methods that can be used with an int type is output as above.

These are the methods that we can apply to an integer in our programming:

(ii)

```
>>>dir(float)
```

Observation:

```
['__abs__', '__add__', '__bool__', '__class__', '__delattr__', '__dir__',  
 '__divmod__', '__doc__', '__eq__', '__float__', '__floordiv__', '__format__',  
 '__ge__', '__getattr__', '__getformat__', '__getnewargs__', '__gt__',  
 '__hash__', '__init__', '__init_subclass__', '__int__', '__le__', '__lt__', '__mod__',  
 '__mul__', '__ne__', '__neg__', '__new__', '__pos__', '__pow__', '__radd__',  
 '__rdivmod__', '__reduce__', '__reduce_ex__', '__repr__', '__rfloordiv__',  
 '__rmod__', '__rmul__', '__round__', '__rpow__', '__rsub__', '__rtruediv__',  
 '__set_format__', '__setattr__', '__sizeof__', '__str__', '__sub__',  
 '__subclasshook__', '__truediv__', '__trunc__', 'as_integer_ratio', 'conjugate',  
 'fromhex', 'hex', 'imag', 'is_integer', 'real']
```

The list of methods that can be used with an int type is output as above.

These are the methods that we can apply to an integer in our programming:

```
>>> dir(str)
```

Observation:

```
['__add__', '__class__', '__contains__', '__delattr__', '__dir__', '__doc__',  
 '__eq__', '__format__', '__ge__', '__getattr__', '__getitem__',  
 '__getnewargs__', '__gt__', '__hash__', '__init__', '__init_subclass__', '__iter__',  
 '__le__', '__len__', '__lt__', '__mod__', '__mul__', '__ne__', '__new__',  
 '__reduce__', '__reduce_ex__', '__repr__', '__rmod__', '__rmul__', '__setattr__',  
 '__sizeof__', '__str__', '__subclasshook__', 'capitalize', 'casefold', 'center',  
 'count', 'encode', 'endswith', 'expandtabs', 'find', 'format', 'format_map', 'index',  
 'isalnum', 'isalpha', 'isascii', 'isdecimal', 'isdigit', 'isidentifier', 'islower',  
 'isnumeric', 'isprintable', 'isspace', 'istitle', 'isupper', 'join', 'ljust', 'lower', 'lstrip',  
 'maketrans', 'partition', 'replace', 'rfind', 'rindex', 'rjust', 'rpartition', 'rsplit', 'rstrip',  
 'split', 'splitlines', 'startswith', 'strip', 'swapcase', 'title', 'translate', 'upper', 'zfill']
```

The list of methods is output as above.

These are the methods that we can apply to a string type in our programming:
